

签到 题解

本题考察选手基本的分析能力，即使没有分析出来也可以写一个数据结构获得较高的分数。

容易发现，如果序列中不存在 0 ， mex 就永远是 0 ，从而此时最大值是 0 ，所有区间都能取到，答案是 $\frac{n \times (n+1)}{2}$ 。

如果存在 0 ，考虑 0 这一个数所对应的子区间，这个子区间权值是 1 ，也容易发现实际上一个序列的权值不可能超过 1 ，因此此时最大值是 1 ，需要考虑如何计算 $mex = m$ 的子区间数，其中 m 是子区间长度。

实际上这相当于 $[0, m-1]$ 这些数恰好在这个子区间中出现过一次，要求的就是这样的子区间的个数。这样的子区间有很好的性质。

最简单的算法是直接扫描所有区间，复杂度 $O(n^2)$ 。

也可以直接扫描线/分治/笛卡尔树分治维护，复杂度 $O(n \log n)$ 。

接下来考察这样的区间的性质：

首先 0 恰好出现一次，而且 0 必须出现，我们可以由此通过序列中的 0 把序列划分成若干段，对每一段分别求解。

此时考虑一个 0 左边的部分和右边的部分，相当于有两个序列 a, b ，你要在两个序列中分别选择一个前缀拼起来得到一个 $[1, m]$ 的连续段（ m 是长度）。

注意到此时最大值是很关键的要素，因为最大值必须和序列长度相等，不妨设最大值在 a 中（否则可以交换 a, b ），枚举 a 的一个前缀，那么此时 b 需要选择的前缀的长度就是固定的。

维护一个指针 l 表示目前 b 选出的长度，容易证明不断枚举 a 的前缀时，所对应的 l 的变化量是 $O(n)$ 的。

（注意此时如果 $a_i > n$ 或者某个数出现了两次就不合法了，需要直接退出）

所以我们可以直接移动指针，然后判断两侧拼起来是否是一个值域连续段，这个过程可以用一个简单的线性数据结构实现。

简单的线性数据结构：维护两个序列 a, b ，每次更改其中一个位置的值，每次询问需要判断 a, b 是否完全相等，因为每次只会更改一个位置，所以直接维护一个 a 和 b 目前不相等的位置的个数即可。

时间复杂度 $O(n)$ 。

实际上也可以对 b 的每个前缀维护前缀集合对应的哈希值进行判断，同样能做到线性。

开挂 题解

题解中描述的部分可能比较繁杂，但整体思路比较简单，阅读题解的时候也可以跳着读。

形式化题意：初始有一个整数 x ，你需要对其进行 n 轮操作，第 i 轮操作会做如下两件事情之一：

1. 令 x 变为 $\max(0, x - a_i)$
2. 令 x 变为 $x + b_i$

你需要对每个 k 求出恰好做 k 次第二种操作和 $n - k$ 次第一种操作最终 x 可以达到的最大值。

首先考察，如果固定了选择操作的策略，有没有什么方法可以快速表示最终的数。

发现实际上是每次对 0 取 $\max$ 比较难处理。

考虑什么时候第一次操作会将 x 恰好变为 0。

假定没有每次对 0 取 $\max$ 这一操作。画一条折线，横轴是轮数，纵轴是 x 的值。然后我们考虑折线上的每个时刻对应的真实的 x 的值。

发现 x 的真实值在某一轮会变为 0，当且仅当这一轮折线上的点是前缀最低点（这个点也就是不高于这个点以前的任何一轮对应的点）。同理我们可以得到， x 的真实值最后一次变成 0，是在折线的最低点对应的那一轮上，然后接下来的每一轮都是直接加或者直接减，不再会出现对 0 取 $\max$ 的操作。

形式化的说，考虑这条折线最低点对应的值，记为 mn ，以及这条折线最后一个点对应的值，记为 s ，最终 x 的值就是 $s - mn$ 。

（注意，此时考虑的是存在某个时刻 x 变成 0 的情况，如果没有变成 0 要特判，但这也是容易的）

由于要最大化 x 的值，所以可以直接钦定 mn 的位置。

钦定最小值的位置可以这样理解：因为要最大化 $s - mn$ ， mn 是最小值，如果钦定到的位置不是最小值，那求出来的值肯定比真实答案要小，所以这样算并不会把答案算错。

考虑我们枚举最小值位置在 p ，可以这样考虑：初始每个位置都是 $-a_i$ ，一次操作把他变成 $+b_i$ ，这样一次操作在折线上体现为给一个后缀 $[i, n]$ ，整体加上 $a_i + b_i$ 。

对于一个固定的 k ，目前的策略肯定是选在 $[p + 1, n]$ 中 $a_i + b_i$ 最大的 k 个位置操作，设前 k 大的 $a_i + b_i$ 的和是 sum ，此时 $s - mn$ 的值就是 $sum - \sum_{i=p+1}^n a_i$ 。

有了这个式子就可以 $O(n^2)$ 的求答案。

考虑不同的 k 之间最优解的关系。

实际上从直观上也能看出来。设 pos_i 为 $k = i$ 时答案取到最大值时 mn 所在的位置，那么 pos_i 是单调的。

证明是很显然的，假设 $pos_i < pos_{i+1}$ ，这意味着 $k = i$ 时取最小值在 pos_i 比在 pos_{i+1} 更优（ pos_i 的定义），考察从 $k = i$ 到 $k = i + 1$ 的过程中实际上只增加了是后缀中第 $k + 1$ 大的数的贡献，后缀 $[pos_i, n]$ 的第 $k + 1$ 大的数显然比后缀 $[pos_{i+1}, n]$ 中第 $k + 1$ 大的数更大，因此此时 $k = i + 1$ 取最小值在 pos_i 这个位置肯定更优，从而矛盾。

因此可以分治求解，设 $solve(l, r, pl, pr)$ 表示尝试求出 $k \in [l, r]$ 的答案，目前已经知道最优解中最小值的位置会在 $[pl, pr]$ 之间，每次取出 $[l, r]$ 区间中点算出其最小值所在位置然后向两边递归。

使用对顶堆/权值线段树维护前 k 小数的和，复杂度 $O(n \log^2 n)$ 。

数数 题解

暴力做法是直接 dp ，可以做到 $O(poly(n))$ ，但是优化起来较为困难。

因此我们需要做一些转化。

首先把 01 序列计数转化为一个折线上的问题。

对于固定的 k ，转化成下面这样一个问题：

计算满足条件的折线的个数，从 $(0,0)$ 开始，每次可以往上或者往右走一步，走到 (n,m) ，并且存在某个 b ，使得这条折线被夹在 $y = x + b + k$ 和 $y = x + b$ 两条直线之间。

容易想到反射容斥。反射容斥解决的是这样的问题：一条折线从 $(0,0)$ 走到 (n,m) ，并且在两条直线 $y = x + b$ 和 $y = x + c$ 之间且与这两条直线没有交点的方案数。

关于反射容斥，如果有不了解的可以看这一篇博客。

<https://www.cnblogs.com/glove/p/18448337>

直接套用反射容斥，容易发现此时 $b \in [-k, 0]$ ，否则 $(0,0)$ 这个点不合法。

可以枚举 b ，然后计算在两条直线之间的折线数量。

此时有可能会算重，因为实际上一条折线可以被很多个 b 的方案对应的直线所包含，但是注意到能包含这条折线的 b 实际上是一个区间，考虑点边容斥。

具体来说，先对答案加上每一个满足 $b = i$ 的折线的贡献，然后再减去每一个 $b = i$ 和 $b = i + 1$ 同时满足的折线的贡献，这样就不会算重。

实际上 $b = i$ 和 $b = i + 1$ 同时满足这个条件相当于这条折线位于 $y = x + b$ 和 $y = x + b + k - 1$ 之间，因此这部分也可以套用反射容斥。

对于一个 k ，做一次反射容斥的复杂度是 $O(\frac{n}{k})$ ，而且需要做 k 次反射容斥，因此此时算一个 k 的复杂度恰好是 $O(n)$ ，总复杂度 $O(n^2)$ 。

考虑优化上述算法，设 $calc(i)$ 表示计算在 $y = x + i$ 和 $y = x + i + k$ 之间的折线的方案数的过程，计算答案时我们会调用一段区间 $i \in [-k, 0]$ 的 $calc(i)$ 。

观察反射容斥的代码，这里先贴一份：

```
1 void mi(int& x,int& y,int b){y-=b,x+=b;swap(x,y);}
2 void mi(int &c,int& b){c = 2*b-c;swap(b,c);}
3 int work(int b,int c){
4     int x = n,y = m,pro = 1,tot = 0;
5     while(x>=0 and y>=0){
6         tot = (tot+pro*c(x)+mod)%mod;
7         mi(x,y,c);
8         mi(b,c);
9         pro = -pro;
10    }
11    return tot;
12 }
```

考虑调用 $\text{calc}(i)$ 和 $\text{calc}(i + 1)$ 在上面的第 k 次循环中的贡献。

发现如果 k 是偶数，两者的贡献的组合数一定相同，如果 k 是奇数，考虑两者贡献的组合数，设其中一个是 C_x ，则另一个一定是 C_{x+1} 。

所以我们可以发现，调用一段区间的 $\text{calc}(i)$ 的这个过程可以并行。

直接维护即可，维护一个区间 $[l, r]$ 表示目前 $i \in [l, r]$, $\text{calc}(i)$ 还在循环中，每次循环可能有 $x < 0$ 或者 $y < 0$ ，此时不合法的一定是一段前缀或者后缀，修改 l, r 即可，每次计算贡献实际上是一段组合数前缀和。

具体部分建议查看 `std`，从而我们对于每个 k 只需要一次反射容斥就可以完成，复杂度 $O(n \log n)$ 。

也存在直接推式子的做法，这里不展开叙述。

这个做法通过直接套用某个算法，然后再观察算法的规律最终快速求解，也是一种可取的方式。

波特 题解

本题主要考察讨论能力和代码实现细节。

我们称一个波特为一个点，一个知道秘密的波特把秘密告诉不知道秘密的波特的过程称为相交，称不知道秘密的波特知道了秘密的过程为激活。

题目中保证了没有点重合，这个性质非常关键。

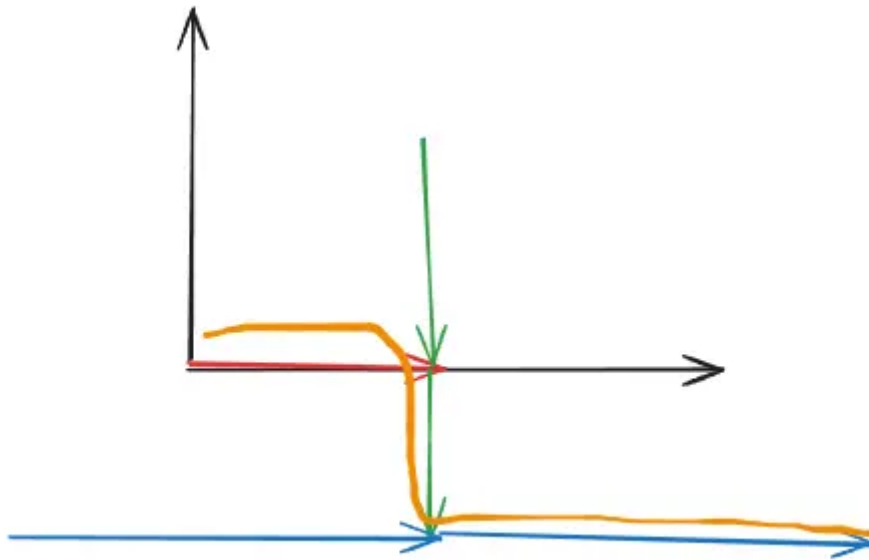
不妨设 1 号点是 $(0, 0)$ 且初始向右，否则可以旋转/平移坐标轴。

我们来考虑某个点 i 初始要朝哪些方向才有可能相交。

不妨先考虑简单的情况，比如 i 初始存在一个方向可以直接与 1 相交，考虑此时 i 能不能通过换个方向然后也和 1 间接相交。

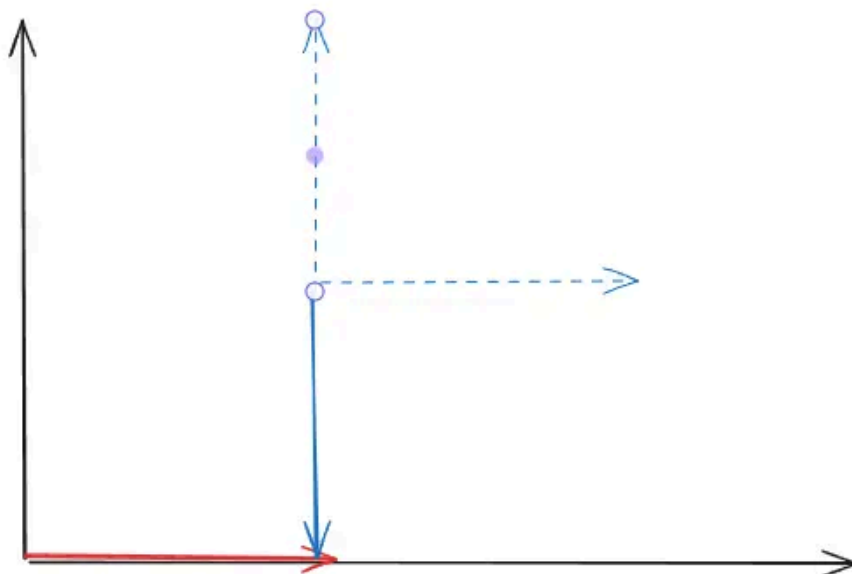
为了方便说明，我们需要先做一个等价的转化，我们把两个点相交，看成 1 号点换了一个方向，这样我们认为就是一条从 1 号点出发的路径再与别的点相交。

请注意本问题实际上还有一个时间轴，并不是在图上线段能相交就代表出现了点的相交。



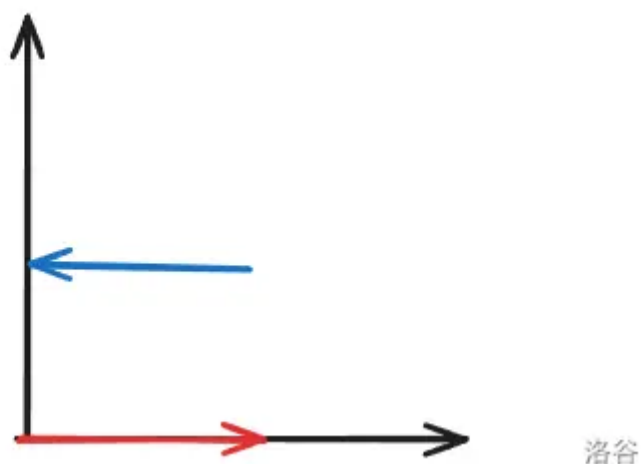
洛谷

如图，红色是一号点，一号点先和绿色点相交，那我们认为它就是往下拐了弯，然后又和蓝色点相交，此时橙色线就是我们认为的等价路径。



洛谷

如图，蓝色点存在一种可以直接和 1 号点相交的方法，很容易发现如果蓝色点沿着两条虚线的方向走，那显然它就会离 1 号点越来越远，那么不可能会存在一条从 1 号点出发到达它的路径



这种情况看上去比较难办。

但是由于我们初始规定一号点向右，而且不存在重合点，所以一号点对应的路径第一步肯定是向右。

实际上如果第一步不向右应该恰好能追上，现在第一步向右了，容易证明，一号点永远追不上蓝色线。

因此此时蓝色点就只能向下，否则不可能有交。

对于其他的点，我们按照 $|x|$ 和 $|y|$ 的关系进行讨论，同样也可以说明每个点最多存在一种初始方向使得它能和别的点相交，而且这个方向可以在一开始就被唯一的确定下来。

接下来的问题只剩下如何模拟这个相交的过程。

设 t_i 表示点 i 第一次激活的时间，这个转移看上去不太容易，我们直接用 **dijkstra** 维护转移。

两个点 $(x_1, y_1), (x_2, y_2)$ 能在某个时刻相遇当且仅当 $x_1 + y_1 = x_2 + y_2$ 或 $x_1 - y_1 = x_2 - y_2$ 或 $x_1 = x_2$ 或 $y_1 = y_2$ 。

我们对于这四种情况分别讨论转移。

我们拿 $x + y$ 举例。

假设目前要转移的这个点 i 是向右的，设它激活的时间为 t_i ，那它要和别的没激活的点相交并将他激活，必须发生在它被激活之后。

也就是说需要满足相交的这个点 $x_j \geq x_i + t$, $x_j + y_j = x_i + y_i$, 实际上只需要满足这一个条件, 根据一开始的论述, 满足这个条件的点在方向上是一定能与目前的 (x_i, y_i) 相交的。

让 (x_i, y_i) 激活 (x_j, y_j) 的时刻也是容易算出来的, 实际上就是 $\frac{|x_i - x_j| + |y_i - y_j|}{2}$ 。

然后直接暴力建出所有边就能获得一个 $O(n^2)$ 的算法。

在一开始将 $x_j + y_j = x_i + y_i$ 这条直线上的按照 x 排序, 那么我们此时会贡献到直线上一个后缀, 而且贡献的点越靠前, 激活的时间越早。

考察 dijkstra 的本质, 我们只需要保证每次取出的是目前还没取出的激活时间最早的点。

设第一个 (x 最小) 能贡献到的点是 p , 我们一开始只更新 p , 因为如果这个更新的贡献生效了第一个被取出来点肯定是 p 。

然后对于每个点 i , 维护从四个方向传过来的最小值。

例如 p 从左侧被激活的最早时间是 t_0 (由 i 贡献而来), 那我们取出 p 的时候将这个 t_0 贡献给 p 在这条直线 $(x_p + y_p)$ 上的后继, 设后继点是 q , 此时的贡献是 $\frac{|x_p - x_q| + |y_p - y_q|}{2} + t_0$ 。

实际上我们是从 i 走到 q 的, 但是我们此时等价的拆分为先从 i 走到 p , 然后给 p 分配一个从左过来的激活时间, 然后再用 p 去更新 q 。(因为在这个方向上 p 比 q 严格更优)

用 set 维护上述过程即可, 代码细节很多, 复杂度 $O(n \log n)$ 。